

CSC 330 Programming Languages

Lecture #1

Data Type Constructs

Classes

By now you should be well aware of what a class is. Classes are what we use to make our own data types and we can create objects from them. But is this the only construct we can use to make an object? Absolutely not! There are structs, interfaces, abstract classes, records, tables, enums, tuples and more. All of these have their purposes and different languages treat these constructs differently. In fact, some languages only have a few of these. Let's start with the familiar class construct. Here is a simple class in Python:

```
# Python
class Movie:
    totalMovieCount = 0

    def __init__(self, title):
        Movie.totalMovieCount += 1
        self.title = title

    @property
    def title(self):
        return self._title

    @title.setter
    def title(self, value):
        self._title = value

m1 = Movie("Titanic")
m2 = Movie("Elemental")
m1.title = "The Titantic"
print(m1.title)
print(Movie.totalMovieCount)
```

This class has a class variable name `totalMovieCount` that keeps track of the total instances of `Movie` that are created. It has an instance variable named `title` that holds the title of the current movie. It converts `title` into a property with a getter and setter (aka accessor and mutator). Finally, we instantiate two movie objects, change the title of one, print out the title that was just changed and print the total movie count. The output is as follows:

The Titantic
2

What would this same exact code look like in Java:

```
// Java
public class Movie {
    public static int totalMovieCount = 0;
    private String title;

    public Movie(String title) {
        Movie.totalMovieCount++;
        this.title = title;
    }
}
```

```

public String getTitle() {
    return title;
}

public void setTitle(String value) {
    title = value;
}

public static void main(String[] args) {
    Movie m1 = new Movie("Titanic");
    Movie m2 = new Movie("Elemental");
    m1.setTitle("The Titanic");
    System.out.println(m1.getTitle());
    System.out.println(Movie.totalMovieCount);
}
}

```

Here we see the exact same code that will produce the same output. Let's look at the differences here.

Python

```

class Movie:
    totalMovieCount = 0

    def __init__(self, title):
        Movie.totalMovieCount += 1
        self.title = title

    @property
    def title(self):
        return self._title

    @title.setter
    def title(self, value):
        self._title = value

m1 = Movie("Titanic")
m2 = Movie("Elemental")
m1.title = "The Titantic"
print(m1.title)
print(Movie.totalMovieCount)

```

Java

```

public class Movie {
    public static int totalMovieCount = 0;
    private String title;

    public Movie(String title) {
        Movie.totalMovieCount++;
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String value) {
        title = value;
    }

    public static void main(String[] args) {
        Movie m1 = new Movie("Titanic");
        Movie m2 = new Movie("Elemental");
        m1.setTitle("The Titanic");
        System.out.println(m1.getTitle());
        System.out.println(Movie.totalMovieCount);
    }
}

```

In Python, class variables are created inside the class but outside of any method. In Java, class variables are created with the "static" keyword and are created in the same area as instance variables. In Python, the constructor is `__init__`. In Java, the constructor is the same name as the class. In Python, the title property is created with `@property` and `@title.setter`. In Java, it is created with `getTitle` and `setTitle` methods. In Python, instantiation looks just like function call. In Java, instantiation requires the "new" keyword. There are more differences, but I think you get the point. You may say that Python does things in a more "normal" way and Java is weird. If you think that, that's probably just because you learned Python first. Perhaps you think Java is

the more “normal” one? This is all subjective, as the truth is that there is NO normal way of doing this! To give you an idea of what I mean, here is the exact same code in multiple other popular languages:

C#

```
public class Movie {

    public static int totalMovieCount = 0;
    public string Title { get; set; }

    public Movie(string title) {
        Movie.totalMovieCount++;
        Title = title;
    }

    public static void Main(string[] args){
        Movie m1 = new Movie("Titanic");
        Movie m2 = new Movie("Elemental");
        m1.Title = "The Titanic";
        System.Console.WriteLine(m1.Title);
        System.Console.WriteLine(
            Movie.totalMovieCount);
    }
}
```

C++

```
#include <iostream>

using namespace std;

class Movie {
public:
    static int totalMovieCount;

    Movie(string title);
    string getTitle();
    void setTitle(string value);

private:
    string title;
};

int Movie::totalMovieCount = 0;

Movie::Movie(string title) {
    Movie::totalMovieCount++;
    this->title = title;
}

string Movie::getTitle() {
    return this->title;
}

void Movie::setTitle(string value) {
    this->title = value;
}

int main(int argc, char** argv) {
    Movie m1("Titanic");
    Movie* m2 = new Movie("Elemental");
    m1.setTitle("The Titanic");
    cout << m1.getTitle() << endl;
    cout << Movie::totalMovieCount
        << endl;
}
```

As you can see, different languages do things differently. There is no “normal” way of doing something. For example, what is the constructor called? Is it the same name as the class, or “__init__”, or “constructor”? Depends on the language. How about properties? Is it @property and @title.setter, or getTitle and setTitle, or Title {get; set;}? Should you use the dot operator or -> to access an object’s members? Where does the main section go? Here is a nice table showing the differences among different languages.

	Ctor name	Properties	Class var
Python	def __init__():	@property def title(): @title.setter def title(value):	class Movie: count = 0
Java	public Movie() {	public String getTitle(){ public void setTitle(String value){	static int count = 0;
C#	public Movie() {	public string Title {get; set;}	static int count = 0;
C++	public: Movie(); Movie::Movie() {	public: string getTitle(); void setTitle(string title);	static int count; int Movie::count = 0;
JavaScript	constructor() {	getTitle(){ setTitle(value){	static count = 0;
Ruby	def initialize()	def title def title=(title)	@@count = 0
Visual Basic	Public Sub New() {	Public Property Title() as String Get Return _title End Get Set(value as String) _title = value End Set End Property	Shared count As Integer = 0

Even something as mundane as comments, function headers and variable creation have different ways:

	Comments	Function Header and Scope	Variable Creation
Python	# ''' '''	def func(): indent	count = 0
Java	// /* */ /** */	type func() { }	int count = 0;
JavaScript	// /* */	func() { }	count = 0;
Ruby	# =begin =end	def func() end	count = 0
Visual Basic	Rem ,	Function func() As type End Function	Dim count As Integer count = 0
Matlab	% %% %{ %}	function output = func() end	count = 0;
Haskell	-- {- -}	func :: type -> type func params = body	count = 0
Pascal	// (* *) { }	function func(); begin end	var count : int; count := 0;

Just like with comments, functions, variable declarations and so on, different languages also have different constructs used to create data types. Classes aren't the only way.

Structs

C, C++, C# and others have something called “structs”. Depending on the language, a struct can vary.

In C, a struct is like a class but it only has data, no methods. This is because C is not an object-oriented language, so while you could technically call an instance of a struct an “object”, it will only have state but no behavior. Here is an example of a struct in C:

```
// c

#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct Fruit {
    char* name;
    int size;
    float price;
};

typedef struct Fruit Fruit;

int main(int argc, char** argv) {
    Fruit* f = malloc(sizeof(Fruit));
    strcpy(f->name, "Apple");
    f->size = 4;
    f->price = 1.15f;
    printf("I have an %s that is %d inches around and costs $%.2f",
        f->name, f->size, f->price);
}
```

In C++, structs help maintain backwards compatibility with C, however they are allowed to contain methods. Actually, there really isn't much difference between a struct and class in C++, aside from the default visibility of its fields¹. Given these two snippets of code:

C++ struct	C++ class
<pre>struct FruitStruct { char* name; int size; float price; };</pre>	<pre>class FruitClass { char* name; int size; float price; };</pre>

In FruitStruct, the fields are public. In FruitClass, the fields are private. This would be equivalent to:

C++ struct	C++ class
<pre>struct FruitStruct { public: char* name; int size; float price; };</pre>	<pre>class FruitClass { private: char* name; int size; float price; };</pre>

¹ Also, the default visibility of inheritance

In my opinion, you should ALWAYS be explicit about the type of visibility your class members have instead of relying on default visibility. If you do, this difference becomes mostly a moot point and structs and classes in C++ can be seen as being very similar constructs.

In C#, however, structs have quite a few differences from classes. Structs form what is known as “value types”. These are data types that have some differences over a “reference type” which is an object of a class. In fact, here is a helpful table that shows the differences:

Value Types (Structs)	Reference Types (Classes)
Can't be set to null	Can be set to null
When copied, value is copied over	When copied, memory location is copied over
Every field must be assigned a value in the constructor ²	No such requirement as any field not given a value will be given a default value automatically
Normally lives on the runtime stack	Normally lives on the heap
Acts more as a primitive value	Is a pointer to the actual memory allocation
Cannot inherit from other structs, but can inherit from interfaces (see below)	Can inherit from other classes and interfaces

Simple data types such as ints, bools, floats, doubles, and chars are all value types. They cannot be set to null, and their value is copied over during assignment. To be clear, instances of structs are still considered “objects” and structs can have methods just like classes. Here is some code that makes this clearer:

```
// C#

struct FruitStructSuper {}
class FruitClassSuper {}

// structs cannot inherit from other structs, so this will NOT compile
struct FruitStruct : FruitStructSuper {
    public string name;
    public int size;
    public float price;

    // must assign a value to ALL fields
    public FruitStruct(string name, int size) {
        this.name = name;
        this.size = size;
        this.price = 1.0f;
    }
}

// classes can inherit from other classes, so this is ok
class FruitClass : FruitClassSuper {
    public string name;
    public int size;
    public float price;

    // don't have to assign value to all fields.
    // price will default to 0.0f
    public FruitClass(string name, int size) {
        this.name = name;
        this.size = size;
    }
}
```

² Actually, as of C# 11, this is no longer the case

```

class Program {
    public static void Main(string[] args) {
        FruitStruct fs = new("Apple", 5);
        FruitStruct fs2 = fs;    // field values are copied over
        FruitStruct fs3 = null; // can NOT do this, will NOT compile.

        FruitClass fc = new("Banana", 5);
        FruitClass fc2 = fc;    // pointer reference is copied over
        FruitClass fc3 = null; // ok to do

        fs.size++;
        System.Console.WriteLine(fs2.size); // fs2's size is unchanged

        fc.size++;
        System.Console.WriteLine(fc2.size); // fc2's size was changed by fc
    }
}

```

For the two lines that won't compile, we get the error message:

error CS0527: Type 'FruitStructSuper' in interface list is not an interface
and

error CS0037: Cannot convert null to 'FruitStruct' because it is a non-nullable value type

Abstract Classes

You should remember abstract classes from CSC 132. Recall that these are classes where at least some of the methods are abstract, meaning they don't have a body and will have to be given one in a subclass. This is denoted in different ways among languages. For example, in Python we had a few different ways of doing this. We could just raise an error:

```

# Python
def foo():
    raise NotImplementedError()

```

Or we could use the ABC module:

```

# Python
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def foo(self):
        pass

```

In abstract classes, some of the methods may be concrete and others may be abstract. So, you could have the following:

```
# Python
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def foo(self):
        pass

    def bar(self):
        print("this method does something")
```

In C# and Java, we use the abstract keyword and omit the function body:

```
// C# (same in Java)
abstract class AbstractClass {
    public abstract void foo();
}
```

Notice that we must put the abstract keyword on the class as well. If the class has any abstract methods, the abstract keyword must be present on the class.

For C++, we create what's called a "pure virtual function". This is equivalent to an abstract method:

```
// C++
class AbstractClass {
public:
    virtual void foo() = 0;
};
```

Recall that "virtual" means it can be overridden in a subclass, which is a requirement for an abstract method since the subclass is the one that provides a body for the method. We put "= 0" at the end signifying that this method is equal to nothing in this class (i.e. doesn't have a body). It's a little weird, but that is the correct way of doing things in C++.

Interfaces

If the class has only abstract methods and no concrete methods, we can use an interface instead. In most languages the only difference between an interface and an abstract class is the fact that an abstract class may contain concrete methods whereas an interface is comprised solely of abstract methods.

In Python and C++, we denote interfaces by making all methods abstract. This means putting `@abstractmethod` above all methods in Python or putting "= 0" after every method header in C++. There is nothing different about the class header.

For example, in Python this is a normal abstract class:

```
# Python
from abc import ABC, abstractmethod

class AbstractClass(ABC):
    @abstractmethod
    def foo(self):
        pass

    def bar(self):
        print("this method does something")
```

Whereas this would be better known as an interface:

```
# Python
from abc import ABC, abstractmethod

class Interface(ABC):
    @abstractmethod
    def foo(self):
        pass

    @abstractmethod
    def bar(self):
        print("this method does something")
```

The difference is subtle and is merely terminology in these languages.

With C# and Java, we have a keyword to denote an interface. In these languages, the keyword "interface" is used instead of "class":

```
// Java
interface Interface {
    public void foo();
}

// C#
interface Interface {
    public void Foo();
}
```

We don't put the word "abstract" on our methods since they are assumed to be abstract because we are using an interface. In fact, up until recently if you attempted to give these methods a body, it would result in a compilation error. However, recent versions of these languages have added support for "default implementation" in interfaces. This muddies the waters on the differences between abstract classes and interfaces and is a controversial feature to say the least. We won't get into that in this class but feel free to look this up online.

Enums

Imagine you are creating a card game. You want to signify the suit that a card can be. You could make the suit a string and type in the name. In Java this would be:

```
// Java
String currentSuit = "hearts";
```

However, this is a very bad approach. There are several potential issues here. One is that if you get sloppy and check the value like this:

```
if (currentSuit.equals("Hearts"))
```

it will never match. You could convert to lowercase:

```
if (currentSuit.toLowerCase().equals("hearts"))
```

But this is slow. In fact, string comparison is slow regardless. This also doesn't stop anyone from doing this:

```
currentSuit = "Hearts Card";
currentSuit = "A Heart";
currentSuit = "heart";
```

While all of these may indicate that the card is hearts, good luck trying to write an if statement for this!

A slightly better approach could be to make it an int and create constants for the different suits. In Java, it would look like this:

```
// Java
final int SUIT_DIAMONDS = 0;
final int SUIT_HEARTS = 1;
final int SUIT_SPADES = 2;
final int SUIT_CLUBS = 3;

int currentSuit = SUIT_HEARTS;
```

This works better since there is no spelling or case issues and an if statement is straightforward:

```
if (currentSuit == SUIT_HEARTS)
```

However, since the variable is an int, we could still do this:

```
currentSuit = 4;
```

Which suit is that? The number 4 doesn't match any suit, so now the currentSuit variable is not referring to any valid suit. Another problem with this approach is that if we want to use this in other classes, we either need to recreate the four constants in that class or make them public and static so we can grab them from another class. But what class do we put these in? They really deserve to go into their own structure. Perhaps if we could put them into a separate structure and then turn that structure into a datatype that we could use on currentSuit. This is exactly what an enum is!

In Java, an enum for this would look like:

```
// Java
enum Suit {
    DIAMONDS,
    HEARTS,
    SPADES,
    CLUBS
}

Suit currentSuit = Suit.HEARTS;
```

The 4 possibilities are in their own datatype known as an enum. We then use this datatype (called “Suit”) to be the datatype for our currentSuit variable. This tells the compiler that the variable’s only valid values are values within the Suit enum. This means we can’t possibly set it to a value outside of this range or a compiler error will occur. It is still easy to check too:

```
if (currentSuit == Suit.HEARTS)
```

This solves all our issues and is a great way to handle this problem. It is very efficient to check the value, it can’t be thrown off by a mistype, it can’t become an invalid value, and any class can refer to any of these four values by simply putting the enum name followed by the value (i.e. Suit.DIAMONDS, Suit.HEARTS, Suit.SPADES, or Suit.CLUBS).

Python enums are a bit different but operate mostly the same:

```
# Python
from enum import Enum

class Suit(Enum):
    DIAMONDS = 0
    HEARTS = 1
    SPADES = 2
    CLUBS = 3

currentSuit = Suit.HEARTS
```

You basically create a class that inherits from the “Enum” class. This adds in special rules to make it behave like an enum. Notice that you still have to give a value to each of the suits. This value can be retrieved using “.value”, like this:

```
# Python
print(currentSuit.value)
print(Suit.SPADES.value)
```

C++ handles enums similar to Java, but with some important differences. The declaration looks the similar:

```
// C++
enum Suit {
    DIAMONDS,
    HEARTS,
    SPADES,
    CLUBS
};
```

The only visual difference in syntax is the semicolon at the end, but a bigger difference is that there is no scope to the values. So, to use the enum values, just simply type their name:

```
Suit currentSuit = HEARTS;
```

C++11 (released in 2011) added support for enum scopes by adding the word "class" to the enum declaration. So, you can change the declaration to:

```
// C++
enum class Suit {
    DIAMONDS,
    HEARTS,
    SPADES,
    CLUBS
};
```

and then use:

```
Suit currentSuit = Suit::HEARTS;
```

Also, in C++ an enum is really just an int in disguise. We can prove this by casting to an int:

```
cout << (int) currentSuit;
```

Even though we don't see this directly, the first value of the enum is 0, the second value is 1, and so on. This means that DIAMONDS is 0, HEARTS is 1, SPADES is 2, and CLUBS is 3. We can actually write this if we want:

```
// C++
enum class Suit {
    DIAMONDS = 0,
    HEARTS = 1,
    SPADES = 2,
    CLUBS = 3
};
```

We can change this too:

```
// C++
enum class Suit {
    DIAMONDS = 4,
    HEARTS = 8,
    SPADES = 37,
    CLUBS = -5
};
```

We can also just give a starting integer value and have it automatically increment from there:

```
// C++
enum class Suit {
    DIAMONDS = 1,
    HEARTS,      // will automatically be 2 since DIAMONDS is 1
    SPADES,      // will be 3
    CLUBS        // will be 4
};
```

C# is very close to C++ here. Enums in C# are really just glorified ints and the value of those ints can be manipulated in the same way as above.

Can we get the same thing in Java? What happens if we take an enum in Java and cast it to an int?

```
// Java
enum Suit {
    DIAMONDS,
    HEARTS,
    SPADES,
    CLUBS
}

public class Enums {
    public static void main(String[] args) {
        Suit currentSuit = Suit.HEARTS;
        System.out.println((int)currentSuit);
    }
}
```

If we try to compile this, we get:

```
Enums.java:11: error: incompatible types: Suit cannot be converted to int
        System.out.println((int)currentSuit);
                           ^
```

In Java, enums are not just ints. They are entire classes. Taking a random object of a class and casting it to an int doesn't make any sense, so casting a value of an enum to an int doesn't make any sense either (at least to Java). If we just print it out with no casting, i.e.:

```
System.out.println(currentSuit);
```

Then the word "HEARTS" gets printed to the screen. This is because the magic `toString()` method has been overwritten for us. But there aren't any numbers associated with this value. If we wanted to put a number to each value, we would have to do something like this:

```
// Java
enum Suit {
    DIAMONDS(0),
    HEARTS(1),
    SPADES(2),
    CLUBS(3);

    public int number;

    Suit(int number) {
        this.number = number;
    }
}

public class Enums {
    public static void main(String[] args) {
        Suit currentSuit = Suit.HEARTS;
        System.out.println(currentSuit.number);
    }
}
```

We have to treat our enum more like a class. We give it an int field and have the constructor set that field. Each value then puts the value for that field in parentheses. When we want to retrieve that field, we treat `currentSuit` as an object and use dot notation to refer to the number field. The reason for all this, is because Java treats this enum as a class. You don't have to write this, but Java essentially looks at this enum and treats it as if it were the following:

```
// Java
class Suit {
    public static final Suit DIAMONDS = new Suit(0);
    public static final Suit HEARTS = new Suit(1);
    public static final Suit SPADES = new Suit(2);
    public static final Suit CLUBS = new Suit(3);

    public int number;

    private Suit(int number) {
        this.number = number;
    }
}

public class Enums {
    public static void main(String[] args) {
        Suit currentSuit = Suit.HEARTS;
        System.out.println(currentSuit.number);
    }
}
```

These two bodies of code are equivalent. Java just adds all of this when you use an enum instead of a class.

Records

A record in general is a data type construct that contains fields and possibly methods. In most object oriented languages, this takes the form of a class. However, it is important to note because some languages treat this as a unique construct.

For example, in the C# language, a record is a class that contains immutable properties and has equality and inequality operators already defined. For example, the code:

```
// C#
public record Person (string FirstName, string LastName);
```

instructs the compiler to create an entire class with FirstName and LastName as read-only properties, having an explicit constructor, overloading the == and != operators, and even overriding the default ToString method so the object is printed out with its custom properties. Essentially, it creates something similar to this:

```
// C#
public class Person {
    public string FirstName { get; init; } // immutable properties
    public string LastName { get; init; } // immutable properties

    // explicit constructor
    public Person(string firstName, string lastName) {
        this.FirstName = firstName;
        this.LastName = lastName;
    }

    // magic toString function
    public override string ToString() {
        return $"Person { FirstName = {FirstName}, LastName = {LastName} }";
    }

    // overloads the != operator
    public static bool operator !=(Person left, Person right) {
        return !(left == right);
    }

    // overloads the == operator
    public static bool operator ==(Person left, Person right) {
        return left.FirstName == right.FirstName &&
            left.LastName == right.LastName;
    }
}
```

We can test this by putting some test code into the entry point:

```
// C#
public class RecordTest {
    public static void Main(string[] args) {
        Person p1 = new Person("Kevin", "Cherry");
        Person p2 = new Person("Kevin", "Cherry");

        System.Console.WriteLine($"p1 is {p1}");

        if (p1 == p2) {
            System.Console.WriteLine("p1 and p2 are the same");
        }
    }
}
```

Java has records as well. They work very similarly to C#, meaning you can create immutable fields (remember Java doesn't have actual properties) and the `toString` method is overridden for you. Java doesn't have operator overloading, but the `equals` method is overridden to give you the same functionality.

```
// Java
record Person (String firstName, String lastName) { }
```

Notice how we don't put `public` in front. Remember you can only have one public type per file. If this was in a file by itself called `Person.java`, then we would put `public` in front. Also, notice that we don't end the record with a semicolon. You must use braces. You can add some more methods and fields here if you wish. If you do add in some fields, they must be marked as static. After typing the record above, here is what the compiler turns this into:

```
// Java
class Person {
    // immutable fields
    private final String firstName;
    private final String lastName;

    // explicit constructor
    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    // accessors
    public String firstName() {
        return firstName;
    }

    public String lastName() {
        return lastName;
    }
}
```



```

// magic toString function
@Override
public String toString() {
    return "Person[firstName=" + firstName + ", lastName=" + lastName + "]";
}

// checks for equality
@Override
public boolean equals(Object other) {
    Person otherPerson = (Person)other;
    return this.firstName.equals(otherPerson.firstName) &&
           this.lastName.equals(otherPerson.lastName);
}
}

```

Once again we can test this out:

```

// Java
public class RecordTest {
    public static void main(String[] args) {
        Person p1 = new Person("Kevin", "Cherry");
        Person p2 = new Person("Kevin", "Cherry");

        System.out.println("p1 is " + p1);

        if (p1.equals(p2)) {
            System.out.println("p1 and p2 are the same");
        }
    }
}

```

Python has something similar, although it isn't technically called a "record". The idea behind a record is to have something that behaves like a class but has immutable fields (or properties) and can be turned into a string (i.e. toString) and tested for equality (i.e. equals). Python has most of these in its "dataclass".

```

# Python
from dataclasses import dataclass

@dataclass
class Person:
    firstName: str
    lastName: str

```

This is meant for classes which only contain data/fields, although Python has no way to restrict the fields to be immutable. Dataclasses automatically have the == operator overloaded as well as the conversion into a string object (which it uses __repr__ for).

Python converts this dataclass into the following:

```
# Python
class Person:
    # explicit constructor
    def __init__(self, firstName, lastName):
        self.firstName = firstName
        self.lastName = lastName

    # magic toString function
    def __repr__(self):
        return f"Person(firstName='{self.firstName}', lastName='{self.lastName}')"

    # checks for equality
    def __eq__(self, other):
        return self.firstName == other.firstName and \
            self.lastName == other.lastName
```

Tuples

Tuples are similar to lists, however tuples are normally immutable. For example, in Python we can have lists:

```
# Python
values = [42, "Hello World", 3.14]
values[0] = 1
print(values)
```

By using square brackets, we create a list. Values within a list can be changed at will and the code above runs just fine. However, switching square brackets with parentheses makes this a tuple:

```
# Python
values = (42, "Hello World", 3.14)
values[0] = 1
print(values)
```

This will produce the following errors when ran:

```
line 2, in <module>
    values[0] = 1
    ~~~~~~^
TypeError: 'tuple' object does not support item assignment
```

As the error states, tuples are immutable and therefore its values cannot change. Tuples can be found in other languages as well:

```
// C++
tuple<int, string, double> values(42, "Hello World", 3.14);
cout << "First element: " << get<0>(values) << endl;
cout << "Second element: " << get<1>(values) << endl;
cout << "Third element: " << get<2>(values) << endl;
// note that there is no set() method available
```

```
// C#
using System;

Tuple<int, string, double> values = new(42, "Hello World", 3.14);
Console.WriteLine(values.Item1);
Console.WriteLine(values.Item2);
Console.WriteLine(values.Item3);

values.Item2 = 43; // Not allowed. Creates error message:
// Property or indexer 'Tuple<int, string, double>.Item1' cannot be
// assigned to -- it is read only
```

Unfortunately, Java doesn't natively support tuples, although there are libraries you can find that offer some support.

Tuples just seem like a collection of values, so why is this included in a lecture on data types?

Named Tuples

Well, with named tuples, we can attach names to the values being held. This gives us something similar to fields and is kind of close to creating a class or a struct.

In Python, it would look like this:

```
# Python
from collections import namedtuple

MyValues = namedtuple('MyValues', ['num', 'msg', 'pi'])
obj = MyValues(42, "Hello World", 3.14)
print(obj.num)
print(obj.msg)
print(obj.pi)
```

So, with named tuples, instead of using an index to refer to the independent values, we can use a name. This is true in C# as well:

```
// C# - approach 1
public class Program {
    public static void Main(string[] args) {
        (int num, string msg, double pi) values = new(42, "Hello World", 3.14);
        System.Console.WriteLine(values.num);
        System.Console.WriteLine(values.msg);
        System.Console.WriteLine(values.pi);
    }
}
```

```
// C# - approach 2
using MyValues = (int num, string msg, double pi); // save as data type

public class Program {
    public static void Main(string[] args) {
        MyValues values = new(42, "Hello World", 3.14);
        System.Console.WriteLine(values.num);
        System.Console.WriteLine(values.msg);
        System.Console.WriteLine(values.pi);
    }
}
```

With regards to C++, there is no native named tuple option. The best you can do is to create a struct (or class with public fields). Then create an object and fill in the field values.

```
// C++
struct MyValues {
    int num;
    string msg;
    double pi;
};

int main() {
    MyValues values { 42, "Hello World", 3.14 };

    cout << values.num << endl;
    cout << values.msg << endl;
    cout << values.pi << endl;
}
```

Note that this way of assigning values to fields only works if those fields are public. For a struct, fields are public by default. You could also use a class here, but you would have to explicitly set the fields to be public.

```
// C++
class MyValues {
public:
    int num;
    string msg;
    double pi;
};

int main() {
    MyValues values { 42, "Hello World", 3.14 };

    cout << values.num << endl;
    cout << values.msg << endl;
    cout << values.pi << endl;
}
```